A Controlled Ablation Study of a Guarded Self-Improvement Loop with a Competitive Global Workspace in a Toy Setting

Abdelkabir Ouadoukou*

November 2026
Technical report (Study 1). Not peer reviewed.

Abstract

We study a small closed loop in which a hypernetwork emits adapters for a worker network, a learnability-driven curriculum proposes tasks, a guarded judge accepts or rolls back each round, and a capacity-limited *competitive workspace* broadcasts selected training statistics back to the hypernetwork. The setting is a synthetic few-shot sinusoid-regression family, run on CPU. We pre-specify four hypotheses and an analysis plan (paired bootstrap, Bonferroni-adjusted intervals, a smallest effect of interest of 0.05 in query MSE) and run four arms (`full`, `no_ws`, `random_ws`, `no_judge`) on ten paired seeds. Results are mostly null or negative. We detect no difference between the competitive workspace and no workspace (H1) or a random-winner workspace (H2); both comparisons have limited power (minimum detectable effects of about 0.07 and 0.06). Removing the judge *improved* final query MSE on the hard evaluation set by 0.086 (Bonferroni 98.3% CI $[-0.168, -0.014]$ on the *arm minus full* difference), contradicting the expectation that the guards help (H3); the judge accepted only about 27% of rounds, so guarded arms received roughly four times fewer updates. A Shapley-style interaction term was slightly larger in magnitude on rejected than accepted rounds (H4, small and confounded). The toy has no adversarial pressure, so these results do *not* show that judge guards are harmful in general. We report the over-conservative judge, a curriculum that collapses to easy tasks, several post-hoc debugging steps and dev-seed diagnostics performed before the report run, and the exact hashes needed to reproduce everything.

1 Introduction and Related Work

Recent work demonstrates pieces of self-improving systems in isolation: agents that rewrite their own code and validate changes empirically [12, 13], systems that mix harness rewrites with weight updates [8], models that generate their own finetuning data [16], proposer/solver self-play with verifiable rewards [14], and hypernetworks that emit adapters from text or prompts [2, 9]. Surveys discuss the wider landscape of recursive self-improvement [3]. Evaluator exploitation is a known hazard: proxy-reward overoptimization [4, 6] and reward hacking of self-play judges [15]. Workspace-style coordination, inspired by Global Workspace Theory [1], has been proposed for LLM agents [10] alongside blackboard architectures [7].

Question. We ask a deliberately narrow, empirical question: in a toy loop where these ingredients are wired together, (i) does a workspace that must *select* what is shared help compared with no

*Independent researcher. Contact: abdelkabir.ouadoukou@gmail.com. This work was carried out with substantial AI assistance; see the disclosure at the end of the paper.

sharing or random sharing, and (ii) do conservative judge controls help or hurt? The contribution is a pre-specified, fully reproducible ablation with honest reporting of null and negative outcomes. It is *not* a claim about recursive self-improvement, language models, or scaling. Every component has prior art; we claim nothing about novelty of components.

2 System Architecture

Task family. Each task is few-shot regression $y = a\sin(wx+p)$ with $a \sim U(0.5, 2.5)$, with $K_{\text{sup}} = 10$ support and $K_{\text{qry}} = 40$ query points. Predicting 0 gives an MSE of about 1.29 ($= \mathbb{E}[a^2]/2$), which is the reference level for "no learning". The worker is a small MLP (hidden size 32) with a low-rank adapter of rank 4.

Loop. Per round t: (1) the curriculum engine C proposes tasks; (2) hypernetwork A emits adapter parameters $\theta_A = (U, V, b_1, b_2)$ from the support set and the workspace broadcast g_t; (3) worker B adapts with 3 inner SGD steps (learning rate 0.05, gradient clipping at 5); (4) A and B are updated by a first-order meta-gradient of the query loss after adaptation (no second-order terms); (5) B and C post summaries to the workspace W; (6) the judge D decides acceptance. On rejection, A, B, C and their optimizers are rolled back to the last accepted snapshot.

Curriculum C. A REINFORCE policy over 5 difficulty bins with learnability reward $4\,s(1-s)$, where s is the inner-loop success rate under threshold $\tau = 0.3$; this peaks at 50% success.

Competitive workspace W. Capacity $K = 4$, ignition threshold $\theta = 0.1$, time-to-live 4 rounds, per-source posting limit 2, typed numeric payloads. Candidates compete on

$$\text{sal} = 0.5\,\text{rel} + 0.3\,\text{nov} + 0.3\,\text{trust} \cdot \text{verified} + 0.2\,\text{util} - 0.3\,\frac{\text{age}}{\text{TTL}}, \tag{1}$$

and the broadcast g_t is the mean of the winners' embeddings, where each embedding is a fixed 16-dimensional random projection of scalar training statistics (a stand-in for a semantic embedding). Consequently the workspace carries little information beyond random projections of scalar training statistics, which is a boundary condition for interpreting H1 and H2. In `no_ws` the broadcast is zeros; in `random_ws` the capacity and TTL are identical but winners are chosen at random. A rejected round retries under the *last accepted* broadcast.

Credit assignment. With old versions (A_0, B_0) and candidate versions (A, B), let $v(\cdot) = -\text{MSE}$ on a fixed diagnostic set and write v_0, v_A, v_B, v_{AB} for the four version combinations. The two-player Shapley values are

$$\varphi_A = \tfrac{1}{2}\big[(v_A - v_0) + (v_{AB} - v_B)\big], \qquad \varphi_B = \tfrac{1}{2}\big[(v_B - v_0) + (v_{AB} - v_A)\big], \tag{2}$$

so $\varphi_A + \varphi_B = v_{AB} - v_0$ exactly (efficiency; 11), and the interaction $I = v_{AB} - v_A - v_B + v_0$ measures co-adaptation. C is excluded because its effect flows through the data distribution.

Guarded judge D. (i) *Sealed randomness*: A, B, C never access D's generator. (ii) *Fresh selection tasks* each round, in the spirit of limiting adaptive reuse of a holdout [5]. (iii) *Lower-confidence-bound gate*: accept only if $\overline{d} - \text{sd}(d)/\sqrt{n} > 0$ for paired per-task improvements d. (iv) *Gap monitor*: flag a round if the query-minus-support gap exceeds $\text{gap}_{\text{ema}} + 0.25|\text{gap}_{\text{ema}}| + 0.02$,

where the EMA is updated only on accepted rounds. (v) *Hidden canary*: a held-out check released as one Laplace-noised bit ($\sigma = 0.02$, margin $0.05\, c_{\text{old}}$, budget 200 queries). A round is accepted only if all controls pass. In `no_judge` every round is accepted; the controls are still logged.

3 Pre-Registration and Experimental Design

Status of the pre-registration. The hypotheses (`docs/hypotheses.md`) were written before the report run and are *informal* (after the run, only the status line, the disclosure section and the deviations log were edited; the pre-run text is kept as `docs/hypotheses_prerun.md`): there is no third-party registry timestamp. Their integrity rests on git commit order (hypotheses and the dev-diagnostics disclosure at commit `<COMMIT-HASH: hypotheses + dev_diagnostics>`; frozen code tagged `frozen-config` at `<COMMIT-HASH: tag frozen-config>`; results at `<COMMIT-HASH: results>`) and on the SHA-256 hashes in Table 1. The stated prior credences in `docs/hypotheses.md` are the author's.

Arms and seeds. Four arms: `full` (competitive workspace, judge on), `no_ws`, `random_ws`, `no_judge`. Report seeds 1–10 (paired across arms); dev seeds 100–104 were used only for diagnostics. Each run uses 12 rounds, 40 steps per round, meta-batch 8. Final evaluations use harness-side sets seeded $90000 + \text{seed}$ that no component sees; per-round curves use sets seeded $91000 + \text{seed}$.

Metrics. Primary: `final_hard` (query MSE on a hard evaluation set, lower is better). Secondary: `final_all` (uniform difficulty), `auc_eval_all` (mean of per-round `eval_all`), and accept rate.

Hypotheses (as pre-specified).

- **H1:** `full` beats `no_ws` on `final_hard` (prior credence of the author: 70% null).
- **H2:** `full` beats `random_ws` on `final_hard` (75% null).
- **H3:** `full` beats `no_judge` on `final_hard` (credences 55/25/20 over help / no difference / hurt, as recorded in the pre-registration).
- **H4:** $|I|$ does not differ between accepted and rejected rounds of `full`, round 0 excluded (65% no difference).

Analysis plan. Paired differences $\Delta = \text{metric(arm)} - \text{metric(full)}$ per seed, with percentile bootstrap CIs (10,000 resamples, RNG seed 0), at the 95% level and at the Bonferroni-adjusted 98.33% level for the three primary comparisons. For lower-is-better metrics, $\Delta < 0$ means the arm beats `full`. Decision rule on `final_hard`: "full helps" only if the Bonferroni CI lies above 0 and the mean $\Delta \geq 0.05$ (the smallest effect of interest, SESOI); "full hurts" if the CI lies below 0 and $-\overline{\Delta} \geq 0.05$; otherwise "no detectable difference", which is *not* evidence of no effect. Minimum detectable effects use $z = 2.394 + 0.842$ with the paired SD. H4 is a single-arm, un-adjusted comparison.

Pre-run development history (disclosure). Full detail is in `docs/dev_diagnostics.md`. (a) The code was debugged on seed 0 before freezing; this led to stability guards (tanh-bounded adapters, inner-loop gradient clipping, loss cap, NaN guard) and to a bug fix in which a rejected round now retries under the last accepted broadcast (before the fix, `full`, `no_ws` and `random_ws` produced identical results because the broadcast only took effect after two consecutive acceptances).

Table 1: Files and SHA-256 hashes of the frozen code and pre-registration used for the report run (`results/config_hashes.txt`). Run started 2026-11-07T12:58:19Z; PyTorch 2.15.0+cu132, Python 3.13.18, CPU, 2 worker processes.

File	SHA-256
`si_loop_toy.py`	`9e4696675fb54046ff3dc04b96c04fce451cc445778cc7be29433c8a0fa6b431`
`run_parallel.py`	`ffa92a41cabc0970db6c867705807847d3eac7add669e626d41d2c236a31e89a`
`docs/hypotheses.md` (pre-run version)	`e2c1d356272b04d3dfb9b39f7545526e8192511315160f87d67a7042665e9764`

Table 2: Per-arm results, mean ± SD over seeds 1–10. Lower MSE is better.

Arm	`final_hard`	`final_all`	`auc_eval_all`	accept rate
`full`	1.048 ± 0.190	0.872 ± 0.148	0.906 ± 0.137	0.267 ± 0.123
`no_ws`	1.038 ± 0.189	0.872 ± 0.166	0.920 ± 0.130	0.275 ± 0.118
`random_ws`	1.022 ± 0.162	0.848 ± 0.154	0.890 ± 0.136	0.275 ± 0.147
`no_judge`	0.962 ± 0.146	0.755 ± 0.092	0.833 ± 0.165	1.000 ± 0.000

(b) Before the report run, an AI assistant ran all four arms on dev seeds 100–104 and a gap-monitor sweep on the `full` arm. The dev-seed outcomes were examined, so the report run is not blind to them. The selection rule for any tuning was declared before the sweep (choose the smallest relative gap threshold with mean dev accept rate ≥ 0.4); no setting reached the target (accept rates 0.17, 0.17, 0.20, 0.23 for thresholds $0.25, 0.5, 1.0, 2.0$ and 0.33 with the monitor off), so *no parameter was changed* and the report run uses the unchanged defaults. (c) The report analysis script was written to implement the pre-registered plan and was cross-checked independently.

4 Empirical Results

All numbers are produced by `analyze.py` from `results/ablation.json` (10 seeds per arm, $n_{\text{diverged}} = 0$ in every arm).

H1 (competitive workspace vs. none): no detectable difference. $\Delta = -0.010$ on `final_hard`, Bonferroni CI $[-0.060, +0.034]$. The interval is compatible with effects in either direction up to about 0.03–0.06 and the approximate minimum detectable effect is 0.068, above the SESOI. This is *not* evidence that the workspace has no effect.

H2 (competition vs. random winners): no detectable difference on the primary metric. $\Delta = -0.027$, Bonferroni CI $[-0.071, +0.015]$; MDE ≈ 0.061. On the secondary metric `auc_eval_all`, `random_ws` was *better* than `full` ($\Delta = -0.016$, 95% CI $[-0.028, -0.005]$, Bonferroni CI $[-0.031, -0.003]$), a secondary, uncorrected-across-metrics finding of unclear meaning that we do not interpret as an effect of workspace content. In one of ten seeds the three judged arms had identical `final_hard`.

H3 (judge vs. no judge): the judge hurt in this toy. `no_judge` had lower `final_hard` by 0.086 (Bonferroni CI $[-0.168, -0.014]$), exceeding the SESOI; by the pre-specified rule the verdict is "`full` clearly worse". The effect is also present on `final_all` (-0.117) and `auc_eval_all`. `no_judge` beat `full` on `final_hard` in 8 of 10 seeds. The mechanism is plausibly that the judge rejected about 73% of rounds ($1 - 0.267$), so guarded arms retained roughly four times fewer accepted updates under an equal round budget; the toy contains no adversarial optimization against

Table 3: Paired differences Δ = arm − `full` (negative means the arm is better than `full`). CIs from a paired bootstrap; "Bonf." is the 98.33% interval.

Metric	Arm	Mean Δ	95% CI	Bonf. CI
`final_hard`	`no_ws`	-0.010	$[-0.050, +0.026]$	$[-0.060, +0.034]$
	`random_ws`	-0.027	$[-0.062, +0.007]$	$[-0.071, +0.015]$
	`no_judge`	-0.086	$[-0.151, -0.025]$	$[-0.168, -0.014]$
`final_all`	`no_ws`	-0.001	$[-0.045, +0.038]$	$[-0.056, +0.043]$
	`random_ws`	-0.024	$[-0.058, +0.003]$	$[-0.068, +0.008]$
	`no_judge`	-0.117	$[-0.192, -0.047]$	$[-0.209, -0.034]$
`auc_eval_all`	`no_ws`	$+0.014$	$[-0.001, +0.031]$	$[-0.004, +0.033]$
	`random_ws`	-0.016	$[-0.028, -0.005]$	$[-0.031, -0.003]$
	`no_judge`	-0.073	$[-0.114, -0.039]$	$[-0.125, -0.031]$

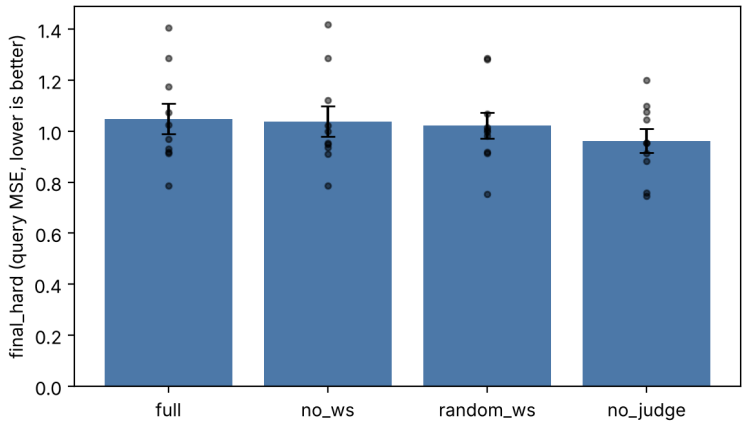


Figure 1: `final_hard` per arm: bars are means with standard errors, dots are individual seeds.

the judge, so the benefit the guards are meant to provide cannot appear. We did not test this explanation.

H4 (Shapley interaction): the no-difference prediction was refuted, weakly. In `full`, with round 0 excluded, 9 of 10 seeds had both accepted and rejected rounds (1 dropped). Mean $|I|$ was 0.0101 on accepted and 0.0164 on rejected rounds (difference +0.0063, 95% CI $[+0.0019, +0.0110]$; signed difference -0.0076, CI $[-0.0133, -0.0020]$), not adjusted for multiplicity. A larger magnitude on rejected rounds is confounded: rejected candidates may simply lie farther from the last accepted model, and the effect size is tiny relative to the loss scale.

Exploratory (not pre-registered). The correlation between φ_A and the judge's lower confidence bound across rounds of `full` was 0.73 (partly mechanical, as both measure improvement of A-containing candidates). Among 23 accepted rounds (round ≥ 1), 2 had worse `eval_all` than the previous round. Mean accepted rounds per `full` seed: 3.2 of 12 (per-seed $4, 1, 3, 2, 6, 4, 2, 4, 4, 2$).

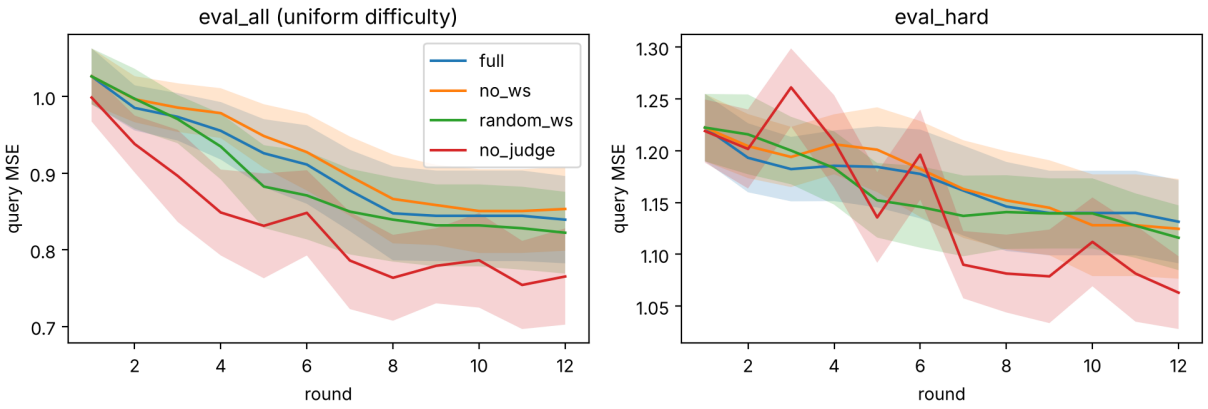


Figure 2: Per-round evaluation curves (mean ± s.e. over 10 seeds) on a uniform-difficulty set (left) and the hard set (right).

5 Discussion and Limitations

Over-conservative judge. On dev seeds the accept rate of `full` was 0.17 (83% of rounds rejected); on report seeds it was 0.267 (≈ 73% rejected). The gap monitor is anchored to the round-0 generalization gap (updated only on accepted rounds), so ordinary learning progress is flagged as overfitting; on dev seeds it flagged about 75% of rounds even in `no_judge`. Even disabling it left a dev accept rate of 0.33, so the lower-confidence-bound gate and the canary are also strict at this sample size. This is a design flaw we chose to report instead of tuning away; a recalibrated judge requires a new pre-registered study.

Curriculum collapse. On dev seeds the curriculum concentrated on easy bins (hard-bin share ≈ 0.02 or lower by the last round), and all arms ended close to the zero-predictor level (≈ 1.29) on the hard set (`final_hard` between 0.96 and 1.05 on report seeds). The toy therefore barely learns hard tasks at this budget, which limits the power of every comparison and of H1 and H2 in particular.

Boundary conditions. (i) Synthetic regression with a non-learned, exact-ground-truth judge: no judge hacking can occur, so H3 says nothing about learned judges. (ii) The workspace carries random projections of scalar statistics, not semantic content; null workspace results here do not transfer to LLM workspaces. (iii) First-order meta-gradients; Shapley attribution covers A and B only. (iv) Ten seeds and low power: confidence intervals are wide relative to the SESOI for H1 and H2. (v) The pre-registration is informal and the dev seeds were inspected before the report run; no tuning followed, but readers should weigh this accordingly. (vi) The word "self-improvement" denotes the loop's design; no open-ended or recursive improvement was observed or claimed.

6 Conclusion and Reproducibility

In a toy loop, a competitive workspace was not distinguishable from no workspace or a random workspace, and conservative judge controls reduced final accuracy relative to accepting every round, mainly because they rejected most updates. The most useful outputs are methodological: an over-strict acceptance gate is itself a failure mode that must be calibrated before judging its benefits; a

pre-specified analysis plan with SESOI and Bonferroni intervals avoided over-reading small differences; and several silent bugs (identical arms due to the broadcast timing, divergence) were caught only by smoke tests.

Reproducibility. Code and data: https://github.com/abdelkabirouadoukou/si-loop. Hypotheses and dev-diagnostics commit: `<COMMIT-HASH: hypotheses + dev_diagnostics>`; frozen-config tag: `<COMMIT-HASH: tag frozen-config>`; results commit: `<COMMIT-HASH: results>`. Verify `shasum -a 256 si_loop_toy.py run_parallel.py` and the pre-run hypotheses (`git show <hypotheses-commit>:docs/hypotheses.md | shasum -a 256`) against Table 1. Reproduce with `python run_parallel.py -seeds 1 2 3 4 5 6 7 8 9 10 -out results/ablation.json` followed by `python -I analyze.py results/ablation.json -out results/`. Bit-exact reproduction may depend on the PyTorch version and platform.

AI assistance disclosure. The code, the analysis scripts, the dev-seed diagnostics, and the drafting of this report were produced with substantial assistance from an AI system (Anthropic's Claude), under the author's direction. The author decided the study design and the publication of null and negative results and is responsible for the content. All numerical values in this paper were checked programmatically against `results/analysis.json`.

References

[1] Bernard J. Baars. *A Cognitive Theory of Consciousness*. Cambridge University Press, 1988.

[2] Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. Text-to-LoRA: Instant transformer adaption. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. arXiv:2506.06105.

[3] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.

[4] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02743.

[5] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015.

[6] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.

[7] Bochen Han and Songmao Zhang. Exploring advanced LLM multi-agent systems based on blackboard architecture. *arXiv preprint arXiv:2507.01701*, 2025.

[8] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self improving AI with harness & weight updates. *arXiv preprint arXiv:2605.27276*, 2026.

[9] Zhiyuan Liang, Dongwen Tang, Yuhao Zhou, Xuanlei Zhao, Mingjia Shi, Wangbo Zhao, Zekai Li, Peihao Wang, Konstantin Schürholt, Damian Borth, Michael M. Bronstein, Yang You, Zhangyang Wang, and Kai Wang. Drag-and-drop LLMs: Zero-shot prompt-to-weights. *arXiv preprint arXiv:2506.16406*, 2025.

[10] Wenlong Shang. "theater of mind" for LLMs: A cognitive architecture based on global workspace theory. *arXiv preprint arXiv:2604.08206*, 2026.

[11] Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games, Volume II*, pages 307–317. Princeton University Press, 1953.

[12] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[13] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026.

[14] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

[15] Chenyu Zhou. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. *arXiv preprint arXiv:2607.05904*, 2026.

[16] Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.10943.